

Tema 1

Introducción



Índice

- ¿Qué es C#?
- Microsoft .NET
- Paradigmas de programación
- Programación orientada a objetos
- Visual Studio
- Creación de tu primer proyecto en C#
- Interfaz de Visual Studio
- Depuración de código



C# (pronunciado *C sharp*) es un lenguaje de programación desarrollado por Microsoft. Fue especificado en el año 2000 y lanzado en 2002 como parte de la plataforma .NET con el objetivo de crear un lenguaje:

- Moderno
- Orientado a objetos
- Seguro
- Versátil

1. ¿Qué es C#?



Moderno

- Se trata de un lenguaje influenciado por otros lenguajes como C, C++ o Java. Es un lenguaje de **sintaxis sencilla**, con una curva de aprendizaje más suave que otros lenguajes.
- Proporciona **mecanismos de alto nivel** que ofrecen al programador una alta abstracción, escalabilidad, portabilidad y reusabilidad.

Esto permite lograr un tiempo de desarrollo rápido sin que ello suponga una limitación en los resultados que podemos obtener.

1. ¿Qué es C#?

Moderno

- Es un lenguaje en **constante evolución**. En cada nueva versión se introducen características, mejoras y aumentos de rendimiento.

Esto hace que el lenguaje de hoy en día (versión 13.0 de la especificación) ofrezca muchos más mecanismos y facilidades de los que existían en las primeras versiones.

- El código de versiones anteriores sigue siendo compatible con las posteriores, pero no al contrario.

1. ¿Qué es C#?

Orientado a objetos

- C# sigue los principios de la **programación orientada a objetos** (POO), permitiendo una programación modular y reutilizable.
- Este paradigma de programación está basado en objetos que contienen información y son capaces de interactuar a través de métodos.
- Entraremos en detalle sobre este paradigma más adelante.

1. ¿Qué es C#?

Seguro

- C# se cimenta sobre la base de ofrecer **software seguro, robusto y duradero** a través de características y mecanismos que ayudan a evitar errores de programación, garantizar la integridad del programa en ejecución y mejorar la gestión de memoria.
- Los principales mecanismos son el tipado fuerte de datos, el manejo automático de memoria y la seguridad en el acceso a ella, el manejo de excepciones o las propias características de seguridad del entorno .NET.

1. ¿Qué es C#?

Seguro

- C# es un lenguaje **fuertemente tipado**. Esto significa que cada variable contiene un tipo de dato específico (entero, decimal, cadena de texto, booleano...) y se verifica en tiempo de compilación.

Esto ayuda a reducir significativamente los errores en el manejo de datos que ocurren en lenguajes débilmente tipados.

- Las conversiones entre tipos de datos se deben realizar de manera explícita y entre tipos compatibles.

1. ¿Qué es C#?

Seguro

- C# utiliza un **recolector de basura** (Garbage Collector) que se encarga de liberar la memoria que ya no se está utilizando de manera automática.

Esto previene errores muy comunes en otros lenguajes como C o C++, donde hay que realizar la gestión de la memoria de manera manual.

- Además, C# restringe el acceso a memoria de manera general (aunque puede realizarse de manera excepcional en entornos '*unsafe*'), garantizando que todos los accesos son seguros y controlados.

1. ¿Qué es C#?

Seguro

- C# implementa un sistema de **manejo de errores** mediante excepciones que permite capturar y manejar errores de manera controlada sin que el programa se bloquee inesperadamente.
- El programador puede definir el comportamiento ante errores específicos o generales, permitiendo gestionar los problemas y recuperar el flujo normal del programa.

1. ¿Qué es C#?

Versátil

- C# es un lenguaje que puede ser utilizado para desarrollar una amplia variedad de aplicaciones en diferentes plataformas.

El mismo código puede ser ejecutado en diferentes entornos y plataformas sin grandes modificaciones, lo que lo hace ideal para proyectos multiplataforma.

- Aunque inicialmente C# estaba enfocado en el entorno de Windows, sus posteriores evoluciones permiten desarrollar aplicaciones de manera nativa en **Windows, Linux y macOS**.

1. ¿Qué es C#?

Versátil

- Además de las diferentes plataformas, la enorme **variedad de aplicaciones** que se pueden desarrollar con C# y la interoperabilidad entre ellas lo hacen uno de los lenguajes más utilizados a día de hoy.
- Podemos encontrar así aplicaciones de escritorio, apps móviles, aplicaciones web, videojuegos, servicios en la nube o aplicaciones IoT desarrolladas con C#.

1. ¿Qué es C#?

Versátil

- **Aplicaciones de escritorio:** se pueden crear aplicaciones gráficas complejas con diferentes tecnologías de interfaz de usuario como Windows Forms o WPF.
- **Apps móviles:** gracias a frameworks como Xamarin y más recientemente .NET MAUI, se pueden desarrollar apps nativas para Android e iOS compartiendo la práctica totalidad del código.

1. ¿Qué es C#?

Versátil

- **Aplicaciones webs:** Con ASP.NET Core podemos desarrollar páginas webs dinámicas, APIs o servicios web y ejecutarlos en cualquier plataforma.
- **Videojuegos:** C# es uno de los lenguajes más de moda en la programación de videojuegos (tanto a nivel comercial como de aficionado) gracias a su integración en el motor Unity, que permite el desarrollo de juegos 2D, 3D, de realidad virtual y de realidad aumentada.

1. ¿Qué es C#?

Versátil

- **Servicios en la nube:** C# se utiliza para el desarrollo de sistemas escalables, microservicios, o funciones serverless tanto en Azure como AWS y Google Cloud.
- **Aplicaciones IoT:** C# también es utilizado para desarrollar soluciones de *Internet de las Cosas* (IoT), permitiendo crear software para dispositivos embebidos y la interacción de este con sensores u otros dispositivos.

1. ¿Qué es C#?

Conclusiones

- C# es un lenguaje sencillo de aprender, muy versátil, que permite un desarrollo rápido de aplicaciones y que ofrece mecanismos de seguridad para evitar errores.
- En el mercado actual, tener conocimientos de este lenguaje permite tener un gran abanico de posibilidades a nivel profesional ya que se trata de un lenguaje muy utilizado en proyectos de índole muy variada.

1. ¿Qué es C#?

¿Qué es .NET?

- .NET es un framework creado por Microsoft que sirve como **marco de desarrollo** para crear aplicaciones.
- Originalmente fue lanzado para Windows de manera propietaria, pero actualmente se encuentra de manera nativa en Windows, Linux y macOS y es gratuito y de código abierto.

2. Microsoft .NET

Historia

- Microsoft lanzó **.NET Framework** a finales de 2001 como una plataforma **exclusiva para Windows**. Este framework tuvo diferentes versiones que van desde la 1.0 hasta la 4.8.
- En 2004 aparece un framework para **Linux** llamado **Mono** y desarrollado externamente a Microsoft. Este framework ofrecía los componentes básicos de .NET y permitía la ejecución de código C# en Linux. Sería comprado por Microsoft en 2016.

2. Microsoft .NET

Historia

- En 2016 aparece **.NET Core**, una versión multiplataforma del framework que supone la implementación en Linux y **macOS** del entorno .NET.
- Además, a lo largo de estos años existieron otras versiones específicas como **ASP.NET** para páginas web, **.NET Micro Framework** para hardware embebido pequeño como microchips, **.NET Compact Framework** para PDAs, **Xamarin/.NET MAUI** para aplicaciones móviles...

2. Microsoft .NET

Historia

- Con el lanzamiento de **.NET 6** a finales de 2021, se **unifican** las diferentes versiones existentes, creando un único framework multiplataforma y de código abierto.
- La versión más nueva, y con la que trabajaremos en este curso, es **.NET 9**. El lanzamiento de .NET 10 está previsto para finales de 2025.

2. Microsoft .NET

Principales características

- Multiplataforma (Windows, Linux, macOS)
- Multilenguaje (C#, F#, Visual Basic.NET de manera nativa, más soporte para otros muchos)
- Soporte para diferentes tipos de aplicaciones: de escritorio, web, móviles, IoT, gafas de RV y RA...
- Incluye una librería base (BCL por sus siglas en inglés) con múltiples funcionalidades implementadas que hace que no empecemos de 0 nuestro proyecto.

2. Microsoft .NET

Arquitectura .NET

2. Microsoft .NET



Biblioteca de clases base (BCL)

- La biblioteca de clases base, o **Base Class Library** (BCL), es un conjunto de tipos, clases, herramientas y APIs que nos proporciona la funcionalidad esencial que se necesita para desarrollar un programa en .NET, como por ejemplo:
 - Salida de datos por pantalla y entrada de datos por teclado
 - Lectura y escritura de ficheros
 - Conexión a base de datos
 - Dibujado de interfaces de usuario

2. Microsoft .NET

Biblioteca de clases base (BCL)

- Esta biblioteca nos **facilitará la programación** ya que gracias a las herramientas que nos ofrece podemos abstraernos del código de más bajo nivel, dejando que .NET se encargue de ello.
- También nos ayuda a crear código más eficiente, seguro y consistente debido a que utilizamos componentes probados, de calidad y refinados a lo largo de muchos años.

2. Microsoft .NET

Biblioteca de clases base (BCL)

- La BCL es una parte fundamental del framework .NET. A lo largo del curso (y durante cualquier desarrollo en C#) utilizaremos numerosas clases y funciones de esta biblioteca.
- Además de estas bibliotecas integradas en .NET, tenemos disponibles una enorme cantidad de bibliotecas que nos ofrecen funcionalidad adicional, tanto oficiales de Microsoft como creadas por otras empresas o por la comunidad.

2. Microsoft .NET

Common Intermediate Language (CIL)

- El Common Intermediate Language es un lenguaje intermedio utilizado por .NET que sirve como puente entre el código escrito en lenguajes de alto nivel (C#) y el código máquina que puede ejecutar el hardware.
- Este lenguaje intermedio permite pues que nuestro código sea independiente de la plataforma, ya que .NET se encargará de convertirlo a código ejecutable dependiendo del destino.
- Para nosotros, el CIL es **totalmente transparente**.

2. Microsoft .NET

Common Language Runtime (CLR)

- El Common Language Runtime es el entorno de ejecución de las aplicaciones .NET, proporcionando los servicios esenciales para la ejecución y gestión de las aplicaciones.
- Es el encargado de compilar el código CIL a código máquina y de gestionar la memoria, el acceso a los recursos del sistema o la integridad de los programas.
- Permite que un mismo código CIL se ejecute en cualquier sistema que soporte el framework de .NET.

2. Microsoft .NET

¿Qué es un paradigma?

- Un paradigma de programación es el enfoque o estilo mediante el que se resuelven los problemas en programación. Los paradigmas definen la manera en la que un programador estructura y desarrolla su código.
- Los lenguajes de programación se clasifican en diferentes paradigmas dependiendo de su filosofía, técnicas y enfoques que ofrecen para controlar la ejecución de los programas escritos en ellos.

3. Paradigmas de programación

Clasificación de los principales paradigmas

3. Paradigmas de programación



Muchas veces, los límites de los paradigmas de programación son difusos. Cada paradigma puede incluir subparadigmas, como hemos visto en la diapositiva anterior.

Además, la mayoría de lenguajes de programación implementan varios paradigmas al mismo tiempo, a veces contrarios entre si.

En el caso concreto de C# podemos decir que es un lenguaje imperativo, funcional, genérico, y orientado a objetos.

3. Paradigmas de programación

Programación imperativa

- La programación imperativa es un paradigma que se basa en el **uso de instrucciones** para modificar el estado de un programa.
- Se enfoca en "**cómo**" se deben realizar las tareas mediante una secuencia explícita de instrucciones o pasos.
- En la programación imperativa debemos controlar el flujo y el estado del programa.

3. Paradigmas de programación

Programación imperativa

- Sus características principales son:
- **Control del flujo:** Utiliza estructuras de control (las veremos en el tema 3) para ordenar la ejecución de las instrucciones.
- **Mutabilidad:** El estado del programa se modifica en el tiempo a través de variables y operaciones que cambian su valor.

3. Paradigmas de programación

Programación imperativa

- Sus características principales son:
- **Secuencialidad:** Las instrucciones se ejecutan por orden de manera secuencial, una tras otra.
- **Uso de variables:** Se almacenan datos en variables que pueden cambiar su valor (entraremos en detalle en el tema 2).

3. Paradigmas de programación

Programación imperativa

- Ventajas:
- **Facilidad de comprensión:** al tener control explícito sobre el flujo del programa y escribir instrucciones concretas, obtenemos un código fácil de entender y analizar.

3. Paradigmas de programación

Programación imperativa

- Ventajas:
- **Alto control:** controlamos todas instrucciones, el orden en que se ejecutan, los recursos que utilizamos...
- **Eficiencia:** Generalmente, la programación imperativa resulta en programas más eficientes en velocidad y memoria que otros paradigmas.

3. Paradigmas de programación

Programación imperativa

- Inconvenientes
- **Complejidad:** debido a la mutabilidad del estado, el mantenimiento del código, especialmente en proyectos grandes, puede volverse muy costoso.
- **Propenso a errores:** la manipulación directa del estado puede llevarnos a situaciones no contempladas o tener efectos secundarios inesperados en otras partes del código.

3. Paradigmas de programación

Programación funcional

- La programación funcional es un paradigma incluido dentro de la programación declarativa. En este tipo de programación nos centramos en **"qué"** se debe realizar, sin detallar los pasos que debemos seguir para lograrlo.

Es lo contrario a la programación imperativa.

- La programación funcional se basa en funciones que devuelven resultados **sin cambiar el estado** ni los datos.

3. Paradigmas de programación

Programación funcional

- Las funciones utilizadas son **deterministas**. Esto quiere decir que para las mismas entradas siempre devuelven las mismas salidas ya que no modifican los datos ni el estado del programa.
- En la programación funcional es típico el uso de **recursividad** en vez de bucles y saltos.
- Los lenguajes puramente funcionales suelen estar reducidos al ámbito académico, aunque algunos lenguajes como C# incluyen características de este paradigma.

3. Paradigmas de programación

Programación funcional

- Ventajas:
- **Código predecible:** al ser código determinista, no tenemos diferentes estados que alteran el comportamiento y por tanto sabemos qué resultados vamos a obtener.
- **Fácil de probar:** al igual que en el anterior punto, el árbol de posibilidades es menor al no tener mutabilidad ni efectos colaterales impredecibles.
- **Abstracción:** permite expresar problemas complejos de manera sencilla.

3. Paradigmas de programación

Programación funcional

- Inconvenientes:
- **Difícil de aprender:** el código es menos natural para nosotros que en la programación imperativa debido a su alta abstracción.
- **Menos eficiente:** el uso de funciones recursivas o el tener que crear nuevas estructuras de datos para no modificar las existentes hacen que la eficiencia de la programación funcional sea menor que la de otros paradigmas.

3. Paradigmas de programación

¿Qué es la programación orientada a objetos?

- La programación orientada a objetos, o POO, es un paradigma que se incluye dentro de la **programación imperativa**. Por tanto, cumple con las características de esta.
- En este paradigma se organiza en base a **objetos** que contienen información (atributos) y comportamiento (métodos). Cada objeto es una ente independiente con su propio estado.

4. Programación orientada a objetos

¿Qué es la programación orientada a objetos?

- Dejamos de centrarnos en la lógica para centrarnos en los **objetos**, que representan entidades del mundo real, y las relaciones e interacciones entre ellos.
- Es el paradigma de programación **más utilizado** en la actualidad debido a que permite la creación de aplicaciones flexibles, escalables, robustas y fáciles de mantener.

4. Programación orientada a objetos

Clases y objetos

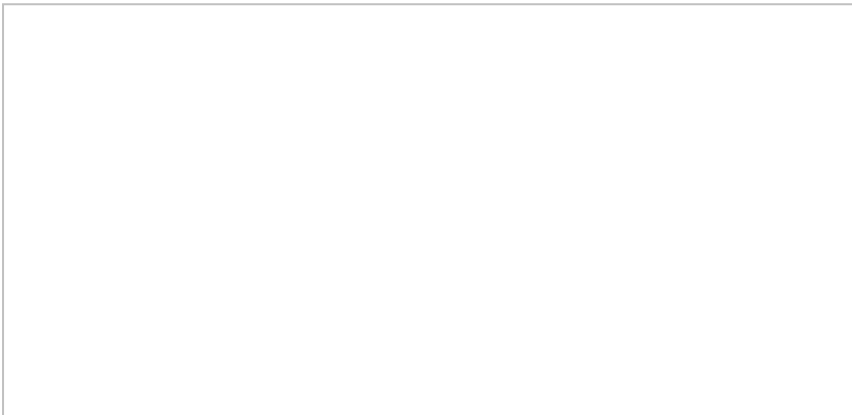
- Las **clases** son las plantillas que definen los atributos y métodos de cada tipo de objeto.
- Los **objetos** son instancias únicas de una clase, cada uno de ellos tiene su propio estado.
- Los objetos pueden **interactuar entre sí**, permitiendo modificar los atributos propios o ajenos.

4. Programación orientada a objetos

Clases y objetos

- En el siguiente ejemplo podemos ver como una clase Persona modela los datos y métodos que poseen los objetos que la instancian, siendo cada uno de estos objetos independientes (cada persona única).
- Cada objeto puede relacionarse con otros y cambiar su estado.

4. Programación orientada a objetos



Principios de la POO

- La programación orientada a objetos se basa en 4 principios fundamentales que marcan el diseño y la implementación de sistemas utilizando este paradigma:
 - Encapsulación
 - Abstracción
 - Herencia
 - Polimorfismo

4. Programación orientada a objetos

Encapsulación

- La encapsulación consiste en agrupar la información de un objeto dentro de si mismo y exponer al exterior únicamente la información que queremos ofrecer.
- Los datos privados no están disponibles para otros objetos, lo que permite garantizar la seguridad e integridad de los datos.
- Al controlar el acceso, evitamos perder el control del estado de un objeto.

4. Programación orientada a objetos

Abstracción

- La abstracción consiste en exponer únicamente los atributos y métodos con los que interactuarán otros objetos, sin importar qué subyace internamente. Puede entenderse pues como una extensión de la encapsulación.
- La abstracción permite simplificar la interacción entre los objetos, ocultando los detalles internos sobre "cómo se hacen las cosas" para ofrecer únicamente los resultados. Esto simplifica el diseño y facilita el mantenimiento.

4. Programación orientada a objetos

Herencia

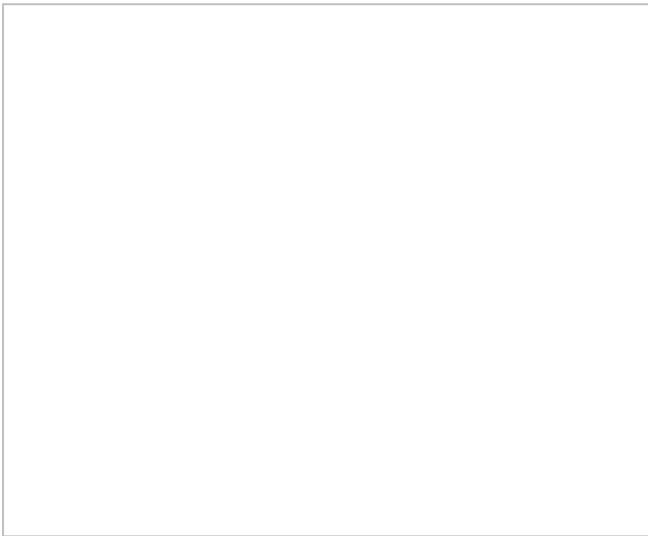
- La herencia significa que se pueden crear clases nuevas a partir de clases existentes.
- Estas clases derivadas poseen los atributos y el comportamiento de sus predecesoras pero extienden su funcionalidad con atributos y métodos adicionales.
- Esto permite la reutilización de código y la extensibilidad de clases mediante jerarquía, evitando repetir comportamientos ya desarrollados, lo que minimiza la posibilidad de errores e inconsistencias.

4. Programación orientada a objetos

Herencia

- En el siguiente ejemplo vemos una herencia de clases. La clase base `Animal` es extendida por las clases derivadas `Ave`, `Reptil` y `Mamífero`, cada una de las cuales posee los atributos de un `Animal` y además añade otros propios.

4. Programación orientada a objetos



Polimorfismo

- El polimorfismo está fuertemente ligado a la herencia y consiste en que cada objeto puede responder de manera diferente a un mismo método dependiendo de su clase.
- En el ejemplo anterior, Ave, Reptil y Mamífero implementan el método Moverse, pero internamente, un objeto ave volaría para moverse, mientras que un reptil se arrastraría y un mamífero caminaría.

4. Programación orientada a objetos

Conclusiones

- La POO es el paradigma de programación más importante para nosotros. Permite construir sistemas complejos gracias a su modularidad, escalabilidad, a la reutilización de código y a su sencillez para abordar problemas complejos.
- A lo largo del curso iremos desarrollando los conceptos que hemos visto y los pondremos en práctica en nuestro código en C#.

4. Programación orientada a objetos

¿Qué es Visual Studio?

- Visual Studio es un entorno de desarrollo integrado (**IDE** por sus siglas en inglés) creado por Microsoft que nos permite desarrollar aplicaciones creadas en .NET.
- Desde Visual Studio se pueden crear todos los tipos de aplicaciones que hemos visto en los puntos de C# y .NET y será **nuestra herramienta de trabajo** a lo largo del curso.

5. Visual Studio

Características

- **Multilenguaje:** Permite el desarrollo de aplicaciones en C#, F#, VB.NET, C++, Java, PHP, JavaScript, Python...
- **Multiplataforma:** Hay versiones de Visual Studio disponibles para Windows, macOS y Linux.
- **Completo:** Ofrece herramientas para realizar todo el ciclo de desarrollo: edición de código, depuración de pruebas, análisis de rendimiento, publicación de aplicaciones, colaborar con otros miembros del equipo... todo integrado en una única herramienta.

5. Visual Studio

Versiones

- La última versión de Visual Studio es 2022. Está disponible en 3 ediciones diferentes:
- **Community:** gratuito para estudiantes, desarrolladores de código abierto y equipos pequeños. Es el que utilizaremos en el curso.
- **Professional:** versión de pago para profesionales y organizaciones con ánimo de lucro de pequeño tamaño.
- **Enterprise:** versión de pago para grandes organizaciones. Ofrece servicios extra de productividad o automatización.

5. Visual Studio

Instalación

- A día de hoy, la única versión compatible con C# 13 y .NET 9 es 2022 Preview.
- Para obtenerla, descargaremos la versión Community desde el siguiente enlace:

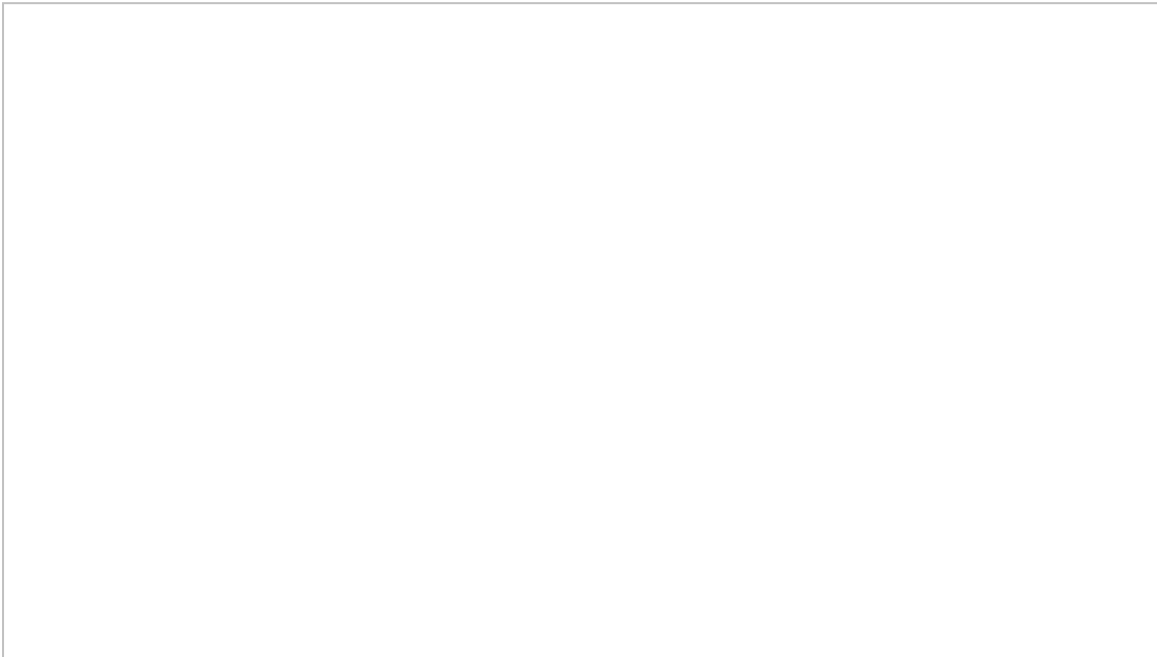
5. Visual Studio

<https://visualstudio.microsoft.com/es/vs/preview/>

Instalación

- En la página, seleccionamos 'Descargar vista previa' y después en la versión Community nuevamente 'Descargar vista previa'.

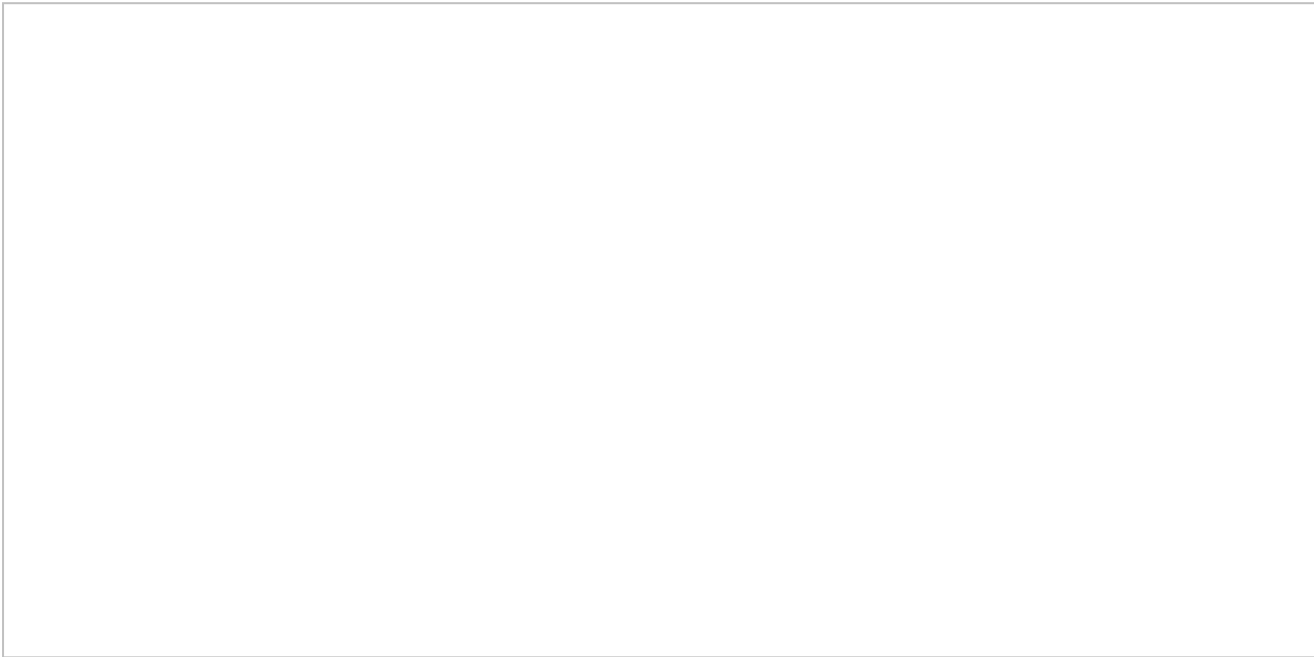
5. Visual Studio



Instalación

- Una vez descargado, ejecutamos, aceptamos los términos y se nos abrirá la siguiente ventana. En ella debemos marcar 'Desarrollo de escritorio de .NET':

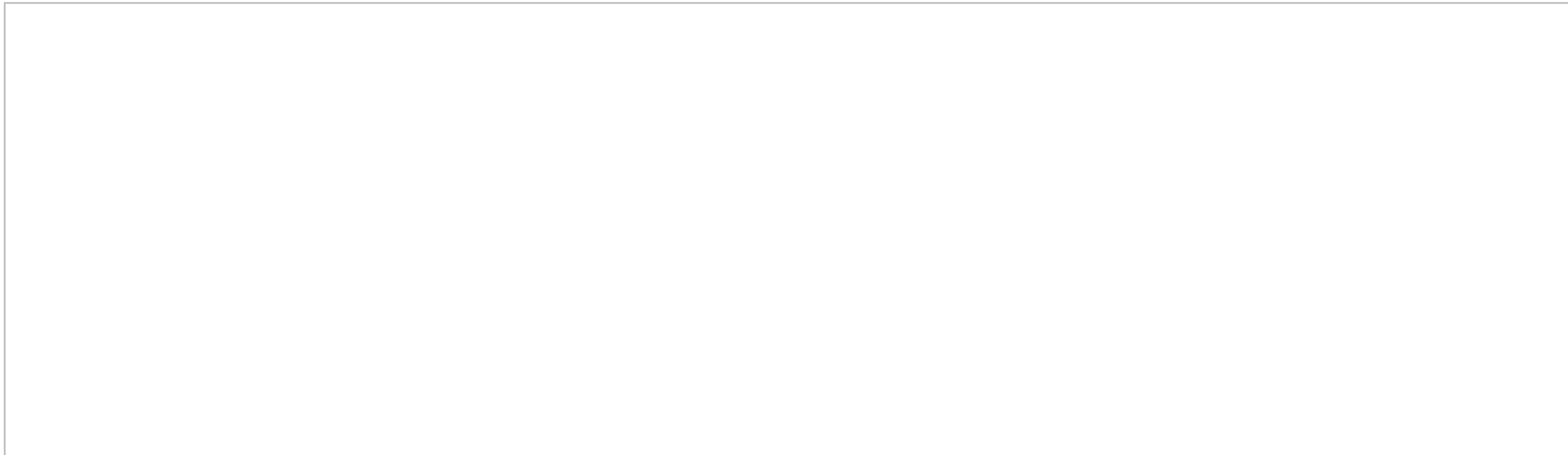
5. Visual Studio



Instalación

- Después de pulsar *Instalar* comenzará la descarga e instalación de Visual Studio:

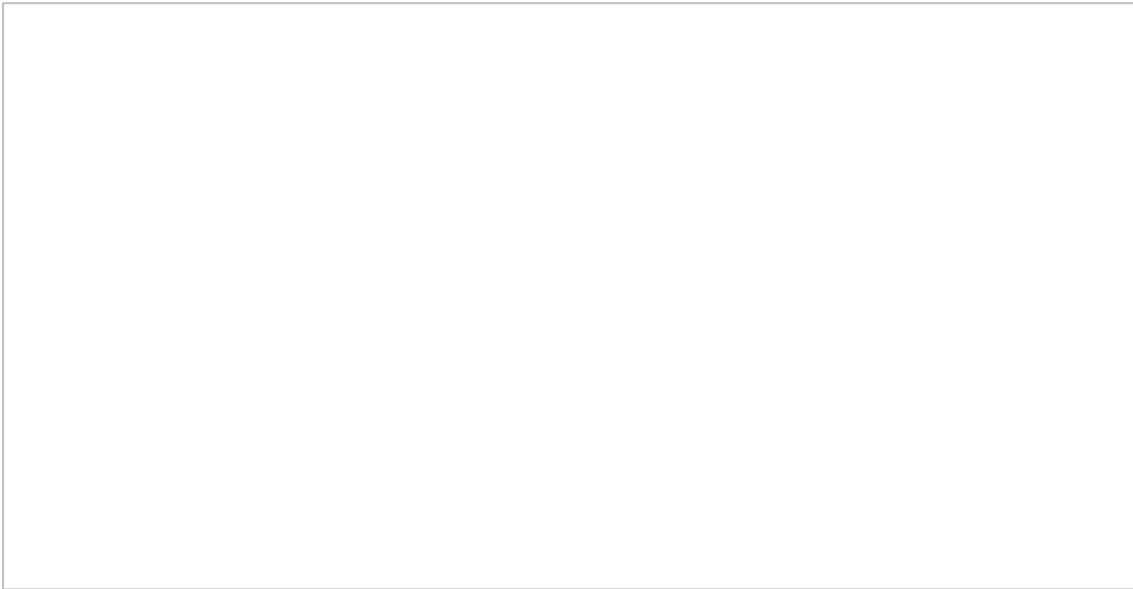
5. Visual Studio



Instalación

- Cuando finaliza la instalación nos aparece el siguiente diálogo para iniciar sesión con una cuenta de Microsoft. Podemos iniciar sesión, crear una cuenta u omitirlo por el momento.

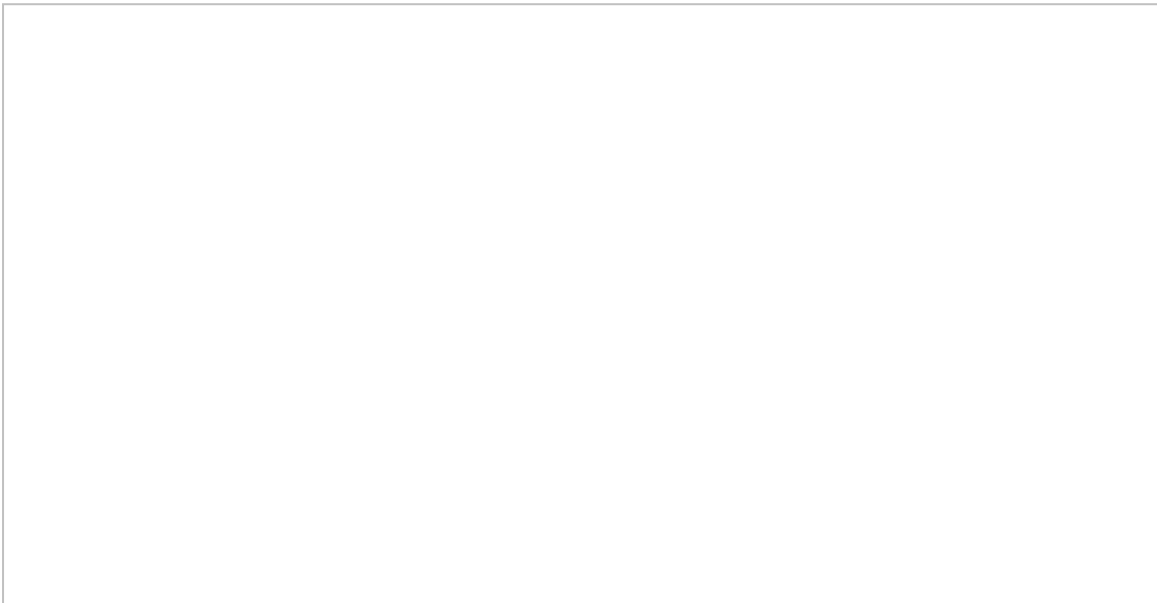
5. Visual Studio



Instalación

- Después nos aparece un nuevo diálogo para elegir tema. Personalmente uso el tema oscuro y todos los vídeos se mostrarán con él, pero podéis elegir el que más os guste. Una vez elegido finalizará la instalación.

5. Visual Studio



Vídeo paso a paso

5. Visual Studio

- Cuando abrimos Visual Studio se nos abre el siguiente diálogo, en el que elegiremos la opción '*Crear un proyecto*'.

6. Creación del primer proyecto en C#



- Se nos abre un diálogo para elegir el tipo de proyecto que vamos a implementar. Elegimos '*Aplicación de consola*' y pulsamos siguiente.

6. Creación del primer proyecto en C#



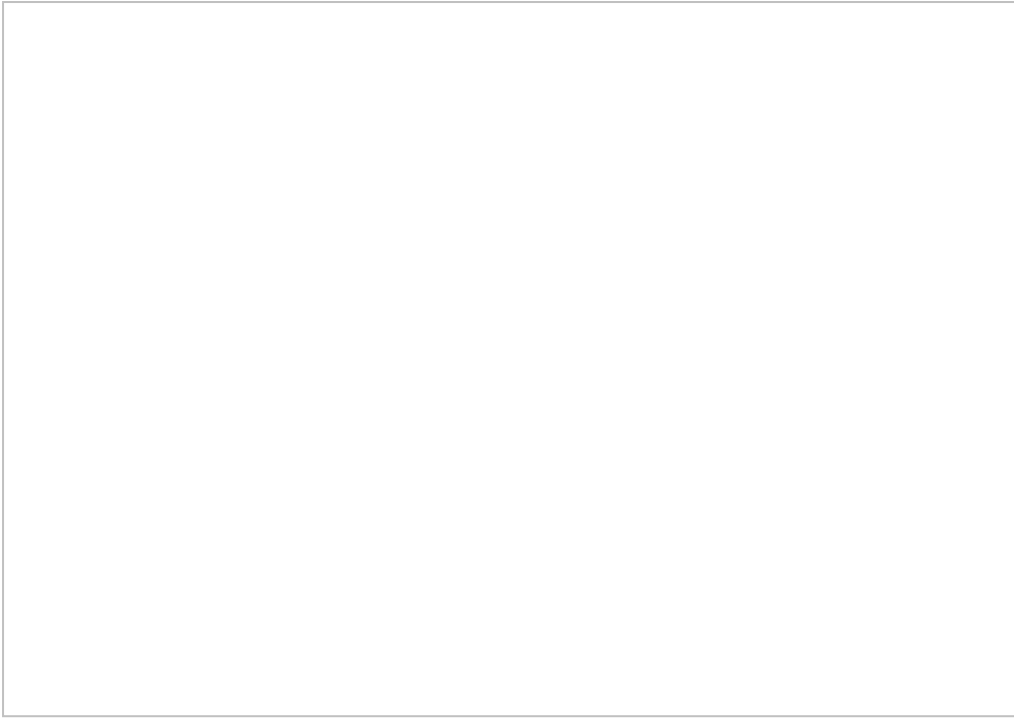
- En la siguiente ventana podremos elegir el nombre de nuestro proyecto y la carpeta en la que se guardará.

6. Creación del primer proyecto en C#



- Por último, tendremos que elegir la versión de .NET. En nuestro caso elegiremos la versión .NET 9.0.
- También vamos a marcar la casilla '*No usar instrucciones de nivel superior*'.

6. Creación del primer proyecto en C#



- Una vez pulsemos crear, se crea nuestro proyecto y aparece la ventana principal de Visual Studio.

6. Creación del primer proyecto en C#



- Visual Studio es una herramienta muy potente que nos permite hacer multitud de tareas dentro de ella. Afortunadamente, aunque su interfaz contiene muchísima información, es bastante intuitiva y nos permite familiarizarnos con ella rápidamente.
- TODA la interfaz es **personalizable**, pudiendo añadir o quitar partes, cambiarlas de tamaño, posición, sacarlas a ventanas flotantes, acoplarlas entre si...
- Vamos a conocer el diseño por defecto y sus partes más importantes.

7. Interfaz de Visual Studio

7. Interfaz de Visual Studio



7. Interfaz de Visual Studio

- **1. Editor:** Es la ventana principal, donde escribiremos nuestro código. Permite tener abiertos múltiples archivos en pestañas. Cuenta con un "ayudante" (*IntelliSense*) mientras escribimos código.
- **2. Panel lateral:** Sección en la que encontramos el explorador de soluciones. Aquí aparecen todos los archivos y componentes que forman parte de nuestro proyecto.

7. Interfaz de Visual Studio

- **3. Barra de herramientas:** Barra con botones separados en diferentes secciones:

operaciones con archivos (nuevo, abrir, guardar...), ejecución de código (ejecución, depuración, selección de configuración) y herramientas auxiliares como comentar o descomentar código, buscar en archivos o añadir marcadores.

- **4. Panel inferior:** Panel con secciones como la lista de errores que tenemos en el código o la salida cuando ejecutamos.

7. Interfaz de Visual Studio

Vídeo explicativo

8. Depuración de código

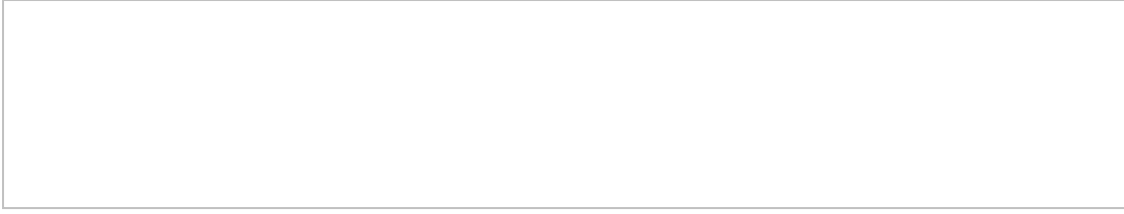
¿Qué es la depuración de código?

- La depuración (o **debugging**, en inglés) es el proceso de encontrar errores o comportamientos no deseados en durante la ejecución de nuestro código.
- Visual Studio nos ofrece una serie de herramientas potentes que nos ayudarán en este proceso.

8. Depuración de código

Depuración en Visual Studio

- Para depurar en Visual Studio, en primer lugar debemos añadir puntos de ruptura o **breakpoints** en las líneas de código que nos interese. Para ello pulsamos en la barra izquierda del editor y nos aparecerá una bolita roja y la línea marcada en rojo.
- Un breakpoint significa que la ejecución del programa que estamos depurando se **parará** cuando llegue a ese punto.



8. Depuración de código

Depuración en Visual Studio

- Para iniciar la depuración, pulsaremos el botón de **Iniciar depuración** en la barra de herramientas superior o la tecla F5 en nuestro teclado.
- El programa comenzará a ejecutarse hasta llegar al primer breakpoint, en el que se detendrá.



8. Depuración de código

Ejecución paso a paso

- Con la ejecución detenida en un breakpoint, podemos continuar la depuración de diferentes maneras:
- **Paso a paso por instrucciones (F11):** el programa continúa y se detiene en la siguiente instrucción.
- **Paso a paso por procedimientos (F10):** el programa continúa ejecutándose y se detiene en la siguiente línea, sin entrar en las llamadas que pueda haber.



8. Depuración de código

Ejecución paso a paso

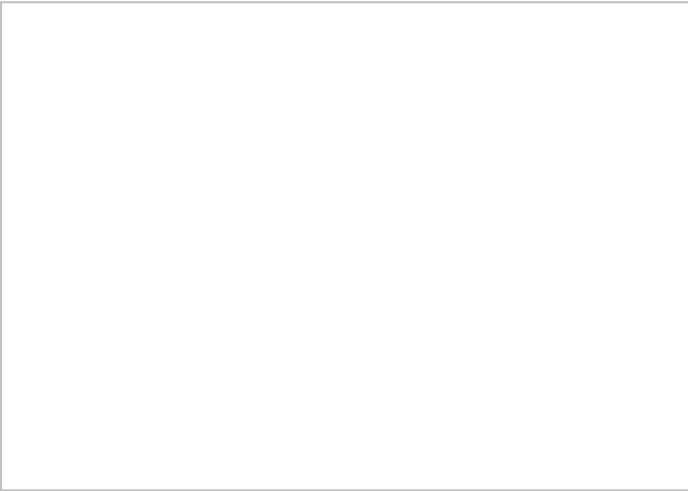
- Con la ejecución detenida en un breakpoint, podemos continuar la depuración de diferentes maneras:
- **Continuar (F5):** El programa continúa ejecutándose de manera normal hasta el siguiente breakpoint.
- **Detener (↑ +F5):** Detiene la depuración y el programa.



8. Depuración de código

Inspección

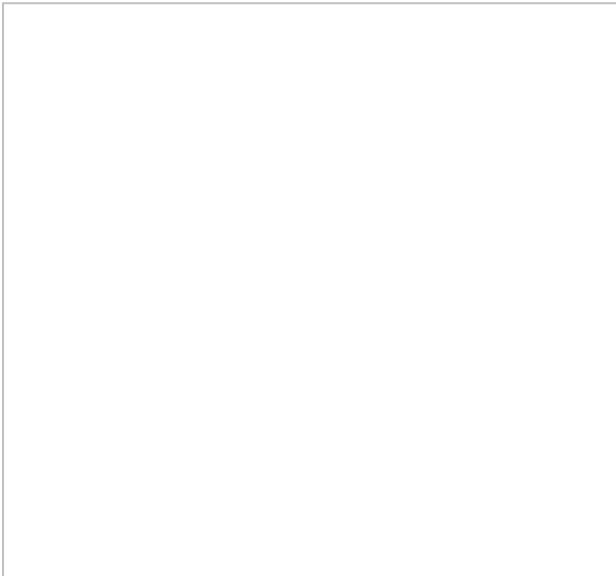
- Cuando nos encontramos detenidos en un breakpoint, podemos inspeccionar el valor de las variables que tenemos en ejecución.
- Para ello, dejaremos el ratón encima de cualquier variable (no hace falta que sea en la línea del breakpoint).



8. Depuración de código

Inspección

- También podemos agregar una variable al inspector de variables mediante *botón derecho* > *Agregar inspección*.
- Esto nos abrirá un panel con las variables inspeccionadas y su valor, que se actualizará en cada breakpoint.





8. Depuración de código

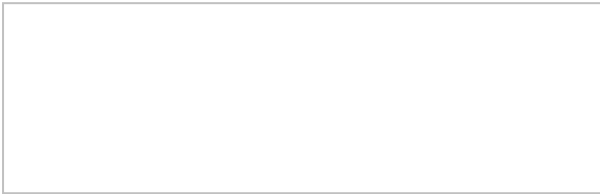
Vídeo práctico

- Para el próximo vídeo vamos a contar con el siguiente código, puedes copiarlo y pegarlo en tu clase Program.cs (no te preocupes si no entiendes lo que hace):

```
1  namespace ConsoleApp1
2  {
3      internal class Program
4      {
5          static void Main(string[] args)
6          {
7              Console.WriteLine("Iniciando depura
8
9              string quixote = "En un lugar de La
10
11             double price = 45.5;
12             double newPrice = Double(price);
13
14             Console.WriteLine(quixote);
15             Console.WriteLine(newPrice);
16         }
17     }
18 }
```

8. Depuración de código

Vídeo práctico



En este tema hemos visto una introducción teórica al lenguaje que vamos a aprender y sobre la programación y sus paradigmas. También hemos instalado la herramienta que utilizaremos en el curso y aprendido a depurar nuestro código.

1. Introducción

Speaker notes